

Content-addressed deduplication: a distributed-storage-system study

CS 4993 · Research specification

Harivansh Rathi

September 2026

Abstract

In a content-addressed store the hash is the address, and an address computed from the bytes is the same on every host. This study builds a content-addressed block backend under unmodified QEMU and measures, on two hosts, what that provides and what the remote cold read costs.

Contents

0	Thesis	2
0.1	Deduplication within a host	2
0.2	What a hash provides across hosts	3
0.3	What it costs	4
0.4	Hypotheses	4
0.5	Outputs	5
0.6	Scope	5
1	Architecture	6
1.1	Components on one host	6
1.2	Write path	6
1.3	Compactor	7
1.4	Read path	7
1.5	Store, index, and manifest	8
1.6	Protocol	8
1.7	Placement and the parameter k	8
1.8	Durability classes	8
1.9	The watermark	9
1.10	Garbage collection	9
1.11	Out of scope	10
1.12	Provenance	10
2	One host	10
2.1	Configurations	10
2.2	Chunk-size arms	11
2.3	Workloads	11
2.4	Metrics	11
2.5	Controls	11
2.6	Prediction from a small census	12

3	Multiple hosts	12
3.1	Two placement modes	12
3.2	Provisioning	12
3.3	Migration	13
3.4	Synchronization after drift	13
3.5	Capacity	13
3.6	Durability classes and their cost	13
3.7	Measurements	14
3.8	The locality objection	14
4	Remote read	14
4.1	Where the time goes in a remote read	14
4.2	Transport probes	15
4.3	Method	15
4.4	Prefetch	16
4.5	Under a guest workload	16
4.6	The FLUSH round trip	16
4.7	RDMA on this testbed	16
4.8	Hypothesis 3, restated	16
5	Plan	16
5.1	Hardware	17
5.2	Schedule	17
5.3	Gates	17
5.4	Descoping order	18
5.5	Risks	18
5.6	Logistics	18
5.7	Future work	18
6	Prior art	19
6.1	Nearest systems	19
6.2	Remote fetch in prior systems	20
6.3	Transport measurements in prior work	21
6.4	Objections already in print	21
6.5	What this study adds	21

0 Thesis

In OpenZFS and dm-vdo the hash of a block is a key in an index that belongs to one pool, and the block is still addressed by its location; only writers to that pool consult the index. In a content-addressed store the hash is the address, and an address computed from the bytes is the same on every host. Three things follow for a fleet: a guest can be provisioned or migrated by moving its manifest, because the chunks the manifest names already exist at their owners; each unique chunk can be stored k times across the fleet instead of once per host; and a chunk can be served from whichever host holds it in memory.

This study builds a content-addressed block backend under unmodified QEMU and measures, on two hosts, what each of the three provides and what each costs. The limit is the testbed: two hosts with static membership, Linux guests, and single-digit terabytes.

0.1 Deduplication within a host

Consider two guests that each run `apt upgrade` and download the same packages. Their disks now hold the same bytes, and no copy-on-write clone can share them, because neither copy descends from the other.

A deduplication table shares them. OpenZFS keeps one per pool, the DDT, and Linux has had dm-vdo in the mainline kernel since 6.9. Both hash every block and share equal blocks at one fixed, aligned size: 4 KiB for dm-vdo, and the volblocksize of a ZFS zvol, 16 KiB by default.

Nearly everything a Linux guest writes is 4 KiB aligned: ext4 uses 4 KiB blocks, partitions start at 1 MiB, and package managers write whole files. Jin and Miller found on VM disk images that fixed-size chunks reach nearly the same deduplication ratio as content-defined chunking (CDC), which places chunk boundaries by the bytes themselves; Liquid measured 77% of bytes removed at 4 KiB fixed blocks on 183 images.

We therefore predict that on one host the backend stores within 10% of what ZFS fast dedup stores at the same block size. Part 1 tests this as hypothesis 1.

None of this reaches across hosts. The DDT is per pool, dm-vdo has no replication, and `zfs send` has not carried a deduplicated stream since OpenZFS 2.0. A fleet of N hosts, each with its own table, stores a chunk shared by all of them N times, and moving a guest to another host sends every block of its image whether or not the destination holds an equal one.

Shared-storage systems deduplicate across hosts by placing every write on the network before it is acknowledged; Ceph RBD with TiDedup is the open example. Page 06 places this study between the two.

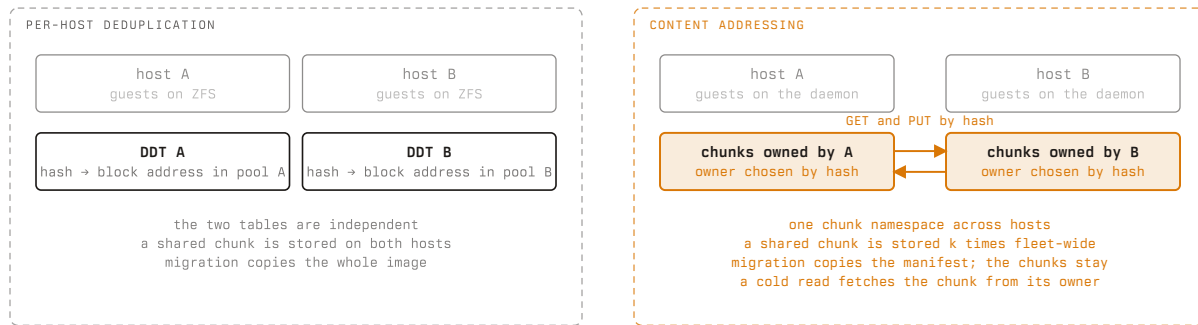


Figure 1: Left: a deduplication table indexes blocks within one pool. Right: a hash is the same name on every host, so placement, transfer, and the cache key follow the content.

0.2 What a hash provides across hosts

A chunk named by its hash has that name on every host, so its placement, its transfer, and its cache key are functions of its content. Each consequence below is a measured claim in parts 2 and 3.

Transfer. Provisioning a guest from an image whose chunks exist in the fleet moves the manifest, at least 32 bytes per chunk, and no chunk data. Migrating a guest moves the manifest plus the bytes written since the last compaction. Migration is measured because it is the operation in which “only unique bytes cross the wire” is easiest to isolate.

Capacity. The fleet stores each unique chunk k times rather than once per host, and each host's index holds entries only for the chunks it owns.

Cache. Every host sends its reads of a chunk to the same k owners, so a chunk that many guests read is hot at its owner. On ConnectX-5 hardware, a 4 KiB read from a peer's memory measured 12 μ s over kernel nvme-rdma and 21 μ s over kernel nvme-tcp (SPDK 24.05), and a 4 KiB read from an enterprise NVMe SSD measured about 80 μ s (Systor '17; blk-switch). **If those figures hold on the testbed, a chunk in a peer's memory arrives before a chunk on the local disk.** Hypothesis 3 tests this.

0.3 What it costs

Three costs come with any post-process deduplicating store, and each is measured rather than assumed: write amplification, because every surviving byte is written to the staging log and again to the store; compactor interference, because compaction shares the guest's disk; and index memory, one entry per chunk in RAM.

One cost belongs to distribution alone: for a chunk this host does not hold, the network is on the read path. Part 3 measures that read over TCP and over RDMA, from the peer's memory and from the peer's NVMe. It then measures prefetch: the daemon reads the manifest, so it knows which chunks a sequential reader will ask for next and fetches them while the guest works on the last one. How much of the cost prefetch removes is a reported number.

One cost is a trade rather than a tax: durability before acknowledgment. A guest's FLUSH means that the bytes must survive. In local class, the default, the daemon acknowledges after `fdatasync` on this host, which is the contract a local disk gives; if the host is lost before compaction has shipped those bytes to their owners, they are lost with it. In fleet class the daemon also sends the bytes themselves to one fixed peer (not the manifest, since the manifest names bytes that exist nowhere else yet) and acknowledges after both this host's `fdatasync` and the peer's; the bytes then survive the loss of this host. Nutanix AOS replicates its write log to another node before it acknowledges, and HPE SimpliVity mirrors every write between two nodes; each product in the page 06 table acknowledges after a network copy. Parts 2 and 3 measure the price of the difference: one round trip and one remote `fdatasync` per FLUSH, over TCP, and over RDMA if that arm lands.

0.4 Hypotheses

Each hypothesis states a metric with its conditions, a comparator, a threshold, the source of the threshold, and what a miss would show. Thresholds are frozen at the end of week 2, after R0 has measured the testbed's `fdatasync` and media times, and do not move after that.

1. Single-host parity. Bytes stored by the backend after compaction and sweep, under the fleet replay at fixed 4 KiB and 16 KiB, are within 10% of the bytes ZFS fast dedup stores at the same volblocksize, and bytes stored in each chunk-size arm are within 10% of the census prediction for that arm. Guest write and read p99 at 4 KiB QD1 are within 20% of a raw file on XFS. The 10% is the alignment argument above plus record headers and the leak between sweeps; the 20% is the passthrough bound of gate G1 plus an equal allowance for the log append and compactor interference. A miss on capture would show that fixed aligned chunks lose duplicates a Linux guest produces; a miss on p99 would show that the daemon, not deduplication, is the cost.

2. Transfer and capacity across hosts. Bytes on the wire to provision and to migrate a guest are within 10% of the manifest size plus the staging tail, against the allocated image size that `zfs send` or `rsync` moves. Bytes stored on both hosts in partitioned mode, after the sweep, are at most 55% of the bytes two per-host ZFS pools hold for the same guests. The transfer bound is the design, since nothing else is sent; the 55% is one copy of the unique set instead of two, because guests cloned from one image give the two pools nearly the same unique set, plus five points for headers, manifests, and the leak. A miss on transfer would mean chunks were sent that an owner already held, a HAS or fence defect; a miss on capacity would mean the two pools shared less than the census predicted, which the census would show first.

3. The remote read. For a 4 KiB read at QD1 whose chunk is not in the local cache: served from the owner's memory, guest-visible latency is lower than the same read served by the daemon from its own NVMe, over TCP and over RDMA; served from the owner's NVMe, it is at most 40% over the local read on TCP and 15% on RDMA; and with reads in flight at or above the bandwidth-delay point, remote sequential throughput is within 10% of local. The thresholds are the literature stack on page 04: about 80 μ s of media, plus 20 to 30 μ s for a userspace daemon over kernel TCP and about 12 μ s for kernel `nvme-rdma`. A miss on the first part would show that the kernel stack or the daemon's wakeup costs more than the media; a miss on the last would show that the fabric, not the depth, bounds throughput.

4. Durability before acknowledgment. Write p99 at 4 KiB QD1 in fleet class is within 3x of local class over TCP, and within 2x over RDMA if the ibverbs arm lands. The window of local class, the seconds between a FLUSH acknowledgment and the durability of those bytes at their owners, is reported as a distribution under the fleet replay. The 3x is one round trip plus one peer fdatsync alongside the local fdatsync, on the page 04 figures and an fdatsync near 40 μ s (NEED DATA; measured on the testbed drive in week 1). A miss would show that the journal path rather than the transport is the cost, and the peer's fdatsync time is reported separately so the two can be told apart.

0.5 Outputs

The system. A content-addressed block backend for VMs under unmodified QEMU on a stock Linux kernel, over kernel TCP, with source, configuration, and the scripts that produce every table.

The single-host table. The backend against ZFS fast dedup and a raw file on XFS: bytes stored, guest p99, write amplification, and index memory, at three chunk sizes. Hypothesis 1 is decided here, and the chunk-size trade is measured here.

The multi-host table. Bytes moved to provision and to migrate a guest, bytes sent to synchronize two drifted guests, fleet bytes stored with one copy per chunk, and index bytes per host, each against what `zfs send` or `rsync` moves and what two per-host ZFS pools hold.

The remote-read measurement. A content-addressed chunk fetched from a peer under a VM block device, at microsecond resolution, over the daemon on kernel TCP and over NVMe-oF on TCP and RDMA, from the peer's memory and from its NVMe, with and without prefetch.

The durability trade. Local class against fleet class on the same hardware: the write latency fleet class costs per transport, and the seconds of acknowledged data local class puts at risk.

0.6 Scope

The study covers hosts that serve guests from local flash, from a homelab up to rack scale; storage arrays and hyperscale economics are out of scope. Hosts hold each other's chunks, which couples the failure domains of compute and storage that shared-storage designs keep apart; the study measures what that costs on the read path and does not model its availability.

Chunks are placed over N hosts by rendezvous hashing: every host scores each (chunk, host) pair with one hash function, and the k highest-scoring hosts own the chunk. Every host computes the same owner set with no shared state, no ring, and no lookup, at N hash evaluations per chunk; CRUSH's straw2 bucket is the same computation with per-host weights (NEED CITE). The testbed is two hosts with static membership, so failure detection, rebalancing, and authentication are out of scope; when a host joins or leaves under rendezvous hashing, only the chunks whose top- k set changes move. One copy per chunk ($k = 1$) on two hosts sends the largest share of cold reads over the network that this testbed can produce, one half in expectation; in general the share is $1 - k/N$. A deployment runs $k \geq 2$ on $N \geq 3$ hosts.

Each image has one writer. Ownership state, the root record that names the writer and carries an epoch, is held on both hosts; two hosts form no quorum, so failover of a lost writer is a scripted decision and not automatic. The study migrates disks only; memory migration is QEMU's.

The guest contract is virtio-blk with a volatile write cache: an acknowledged FLUSH is durable and nothing else is. Every configuration runs with QEMU `cache=none`, so the host page cache is bypassed everywhere.

Equal BLAKE3 hashes are taken to mean equal bytes; a sample of matches is verified byte for byte and the sample size is reported. The store is trusted infrastructure, so deduplication side channels are documented and excluded.

Experiments run at single-digit TB, and larger figures are projections from measured constants, labeled as such. RDMA is a measurement arm on page 04; nothing in the architecture requires it.

1 Architecture

In local class the network is on the read path only, and only for a chunk this host does not hold. In fleet class it is also on the FLUSH path, once per FLUSH, to one fixed peer. No other message precedes an acknowledgment.

1.1 Components on one host

The guest sees a virtio-blk device on stock QEMU. QEMU connects it over vhost-user-blk to one process per host, the daemon, and all new code lives there. Guest memory is shared with the daemon, so requests are read in place, and storage IO goes through io_uring. The device advertises a 4 KiB logical block, so every write is a whole number of blocks and no read-modify-write exists.

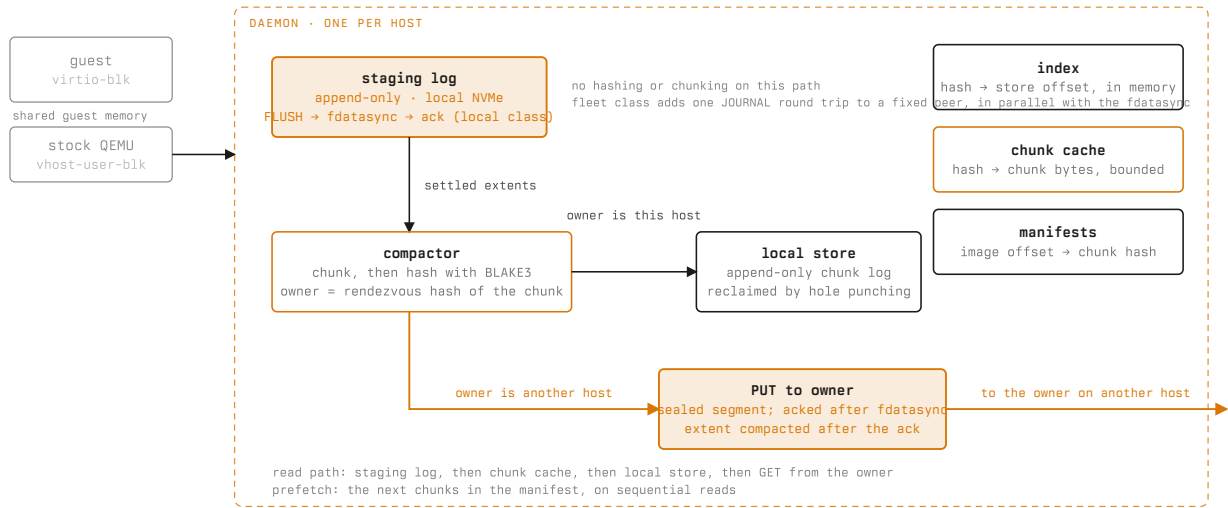


Figure 2: One host. The write path ends at the staging log. The compactor is the only component that talks to other hosts on the write side, and it does so after the acknowledgment.

1.2 Write path

Guest writes append at block granularity to a staging log on local NVMe. Every append is stamped with a per-image sequence number inside the same critical section as the append, so log order is sequence order and replay preserves last-write-wins. FLUSH is fdatsync of the log followed by the acknowledgment, and it covers the highest sequence number completed on any queue of the device, because virtio-blk has no FUA (force unit access) and requests arrive on several queues. If the guest negotiates neither VIRTIO_BLK_F_FLUSH nor a writeback cache, every write is acknowledged after fdatsync. The hot path hashes nothing and chunks nothing, so a large write proceeds at sequential-append speed.

Durability comes from the log alone: every file is opened O_DIRECT, so the page cache never holds the only copy of anything. A waiting FLUSH starts an fdatsync at once, FLUSHes that arrive during one are covered by the next, and an idle fdatsync every 50 ms upgrades writes no FLUSH has asked for. There is no timed linger: in a prior implementation by the author, a 500 μ s linger in front of an fdatsync averaging 35 μ s was unproductive 97% of the time (unpublished measurement). Fresh data is read back from the log at the cost of one NVMe read, the cost R0 pays; an in-memory map from block to log offset supplies the location.

The staging log is finite, so a governor paces compaction on the measured drain rate, with an idle trigger so that nothing sits in staging after a workload ends. When ingest outruns compaction the guest sees added latency and never an error, because virtio-blk has no out-of-space status and an IO error shuts down the guest's filesystem. The point where pressure engages, and the latency it adds, are both measured.

1.3 Compactor

A background pass reads settled extents from the staging log, cuts them into chunks, hashes each with BLAKE3, and skips any hash that every current owner already holds and has fenced; a copy in a cache does not count. Chunking is fixed 4 KiB, fixed 16 KiB, or FastCDC with boundaries snapped to 4 KiB, chosen per arm on page 02. Settled means unwritten for a settle window, so an extent overwritten inside the window is chunked once, in its final form; the window is a parameter and its effect on chunk traffic is measured. A discarded or zero-filled range is one range entry in the manifest that names no chunk, so DISCARD and WRITE_ZEROES of any size consume no store space and constant compactor work. Deferred hashing behind a write buffer is Liquid's design and Fossil's before it; the buffer here is a durable log with a FLUSH contract rather than memory flushed at shutdown.

For each new chunk, the owner set is the first k hosts in rendezvous order of its hash. If this host is an owner, the chunk is appended to the local store and made durable with `fdatasync`. Otherwise it goes to each owner in a sealed segment of many chunks, which the owner appends, `fdatasyncs` once, and acknowledges. **Only after every owner's acknowledgment does the extent count as compacted.** If an owner is unreachable, the compactor appends the chunk to the local store as a surplus copy, pinned until that owner acknowledges it later; the staging log therefore trims on local durability alone, a peer outage costs the guest latency and nothing else, and the sweep reclaims the surplus after the acknowledgment. A chunk the compactor has produced stays pinned, in the staging log or in a store, until the manifest commit that references it is durable, and an owner never reclaims a chunk it acknowledged before that fence. The staging log is therefore the write-ahead log for every chunk this host produces, wherever the chunk ends up.

Two costs come with this design, and both are measured. Every surviving byte is written at least twice, staging then store, plus journal traffic. Compaction reads and writes the same device the guest is using, so guest p99 is measured with the compactor active and idle. The compactor holds no lock the FLUSH path takes, and a test slows the store to one second per append and checks that every FLUSH still completes within its budget.

CDC over a dirty extent re-chunks from the last settled boundary before it to the first boundary after it that agrees with the existing cut. This is the standard resynchronization rule (LBFS locality; Xet's boundary reset), and it is why CDC never runs on the hot path: one aligned write can move every boundary in its neighborhood.

1.4 Read path

A read checks the staging log, then the chunk cache, then the local store if this host holds the chunk, and otherwise sends GET for the hash to an owner. The owner answers from its cache if the chunk is hot and from its store otherwise. Every chunk that arrives over the network is hashed before it is used, so a wrong or corrupt reply is detected and never served. Fresh data is served without indirection; settled data incurs the manifest lookup, the index lookup, and, if the chunk is remote, one round trip. GET runs on its own connections with priority over PUT and over compaction IO at the serving disk, so a guest-blocking read never waits behind a bulk transfer.

The chunk cache is daemon-owned memory keyed by hash, LRU, with a size that is a parameter. A fetched chunk this host does not own lives in that memory cache only. Liquid persisted fetched blocks in an on-disk copy-on-read cache; here a refetch from a peer's memory is predicted at 20 to 30 μ s against about 80 μ s for a local disk hit, so a disk tier would pay only for chunks that are cold at their owner too. The disk tier is a knob, noted, and measured only if time remains; the residual cost of one copy per chunk on page 04 is measured without it. Because every file is O_DIRECT, the kernel page cache holds nothing on any host, and the cache size is set equal to the ARC (adaptive replacement cache) limit on the ZFS configuration.

Prefetch is the daemon issuing the next D hashes from the manifest in one GET when it sees sequential reads, and optionally replaying a recorded boot profile. The guest's own readahead is left at its default and adds to D . D is swept on page 04.

1.5 Store, index, and manifest

The local store is an append-only log of records (length, hash, checksum, bytes) and is authoritative for the chunks this host owns. The index maps hash to store offset, lives in memory, and is rebuilt by scanning the store without re-hashing, because the hash is inline; its bytes per TB is the constant the chunk-size arms measure. In partitioned mode a host indexes only the chunks it owns, so per-host index memory is k/N of the fleet's. The index is written only after the data it points to is durable, at every fence. The manifest, one per image, is a journaled tree from disk offset to chunk hash, packed in offset order. It lives with the guest's host and moves when the guest does.

1.6 Protocol

Message	Reply	Used by
GET(hashes)	bytes per hash	cold read, prefetch
PUT(segment)	ack after one fdatasync	compactor sending a sealed segment of chunks to an owner
HAS(hashes)	bitmap of hashes the owner lacks or has not fenced	compactor before PUT, so only missing chunks are sent; provisioning verification
LIVE(epoch, hashes)	ack	garbage collection
JOURNAL(image,ack after fdatasync range)		fleet class: the staging tail to the fixed journal peer on FLUSH

Messages are length-prefixed over kernel TCP with `TCP_NODELAY`, driven by `io_uring`. GET and JOURNAL have their own connections and priority; PUT is bulk. Every message is idempotent and named by hash or sequence number, so any of them can be retried. The daemon runs busy-polling or blocking; page 04 measures both, because the scheduler wakeup is part of the cost. Rendezvous hashing means a reader already knows the owners of every hash, so no host looks up another's index.

RDMA and NVMe-oF exports appear on page 04 as probes that show what the kernel stack costs; the architecture depends on neither.

1.7 Placement and the parameter k

The owner set of a chunk is the first k hosts in rendezvous order of its hash. The journal peer for fleet class is not chosen this way: a journal needs a fixed home with ordered replay, so each image names one peer at creation and keeps it. k is the one multi-host parameter. With N hosts, $k = N$ places every chunk on every host (replicated) and $k = 1$ places each chunk on exactly one (partitioned); on the two-host testbed these are $k = 2$ and $k = 1$. Page 03 measures both, and a deployment would run $k \geq 2$ on $N \geq 3$ hosts.

1.8 Durability classes

Durability is a per-image class on one pipeline; the class changes who waits at FLUSH and for how long, and nothing about where bytes end up. **Local class**, the default: FLUSH returns after fdatasync of the staging log on this host. **Fleet class**: the staging tail since the last FLUSH is sent to the image's journal peer, which appends it to its own log and fdatasyncs; the send proceeds in parallel with this host's fdatasync, FLUSH returns after both, and FLUSHes from several images to the same peer share one round trip and one fdatasync. Local class is the contract a local disk gives, which is why it is the default against R0 and R1. Fleet class is what Nutanix AOS and HPE SimpliVity do before they acknowledge, and page 03 measures what it costs.

Failure	Local class	Fleet class
daemon crash	nothing acknowledged is lost: replay the staging log from D, re-run compaction	same
host crash, power loss	nothing acknowledged is lost: FLUSH was fdatasync on local NVMe	same
host lost	acknowledged bytes not yet durable at an owner, exactly (O, E], are lost; R0 and R1 lose everything	the staging tail survives: the journal peer replays (D, E] onto a new host; chunks the lost host owned survive only if $k \geq 2$, as in the row below
peer lost, $k = 1$	chunks it owned are unreadable until it returns, and lost if its disk is; a read that needs one waits or fails with an error, never returns stale bytes; writes continue, with surplus copies standing in	

Two rules hold in both classes. The compactor never sends a chunk whose staging extent is not yet durable on this host. Transfer is two-phase: the owner fdatasyncs and acknowledges before the sender marks anything compacted or reclaimable.

1.9 The watermark

Every image carries three integers. E is the highest sequence number with no unconfirmed append before it; in local class confirmed means on local NVMe, in fleet class it means on the journal peer too. D is the highest sequence number whose chunks are durable in a store, at their owners or as surplus copies on this host, and whose manifest entries are committed. $O \leq D$ is the highest sequence number whose chunks are durable at every owner; O equals D except while a surplus copy stands in for an unreachable owner. FLUSH waits for E. A snapshot cuts at E. The staging log is trimmed below D, and trimmed regions are discarded so the drive does not copy dead bytes. Recovery and migration replay exactly (D, E]. A lost host loses (O, E] in local class. E never skips a hole, because a maximum over confirmations forgets the append still in flight, and that is the answer that loses acknowledged data.

Two logs, staging and the manifest journal, must agree after a crash, and staging is senior. Re-running compaction over the replayed extents yields a manifest whose every offset maps to the same bytes; it need not yield the same chunk boundaries under CDC, and the sweep reclaims the orphans of the first run. `kill -9` at any point, then this replay, must pass `fiio --verify` before any number from the daemon is reported; the log's torn tail is tested in both shapes, a shortened file and a partial record followed by preallocated zeros. Three more cases have tests because each is a defect the author met in a prior implementation: an empty discard that acknowledged a sequence number nothing wrote and wedged the next FLUSH; a FLUSH that must cover writes completed on any queue, checked with a multi-queue test and a negative control that shows the test can see the reordering; and a daemon that stops answering, which leaves the guest in D-state because virtio-blk installs no timeout handler.

1.10 Garbage collection

A chunk is live if any manifest on any host references it, or if an in-flight compaction has pinned it; a copy in a cache is never a reference. Each host sends each owner the live set for an epoch with `LIVE`, and the owner sweeps with `FALLOC_FL_PUNCH_HOLE` over dead records; there are no reference counts. Liquid ran the same mark-and-sweep with Bloom-filter live sets over its data servers. ZFS frees an overwritten block the moment its reference count drops; this design does not, so space leaks between sweeps. The sweep therefore runs before every capacity measurement, and the bytes it reclaims are reported beside the capacity number as the leak; concurrent collection is out of scope.

1.11 Out of scope

Membership changes, failure detection, rebalancing when a host joins or leaves, authentication and encryption on the wire, measurement on more than two hosts, and concurrent garbage collection. Each is named in future work on page 05, and none affects a number this study reports.

1.12 Provenance

Component	Source	License
hypervisor	stock QEMU, unmodified, vhost-user-blk front end	GPL-2.0
vhost-user protocol	rust-vmm vhost-user-backend, vm-memory, virtio-queue; Cloud Hypervisor's vhost_user_block read as reference	Apache-2.0 / BSD-3-Clause
hashing	blake3 crate	CC0 / Apache-2.0
chunking	fastcdc crate	MIT
host filesystem	XFS on the dedicated NVMe, O_DIRECT, hole punching; ZFS never sits under the daemon	
staging, watermark, governor, compactor, store, index, manifests, cache, protocol, journal peer, garbage collection	this study	new code

Because the hypervisor is unmodified, no result can be an artifact of a patched QEMU, and the raw-file control runs the identical binary.

2 One host

Part 1 runs the same stock QEMU, the same guest, and the same NVMe device, and varies only the storage behind the device. We predict a tie on capture between the backend and ZFS fast dedup. The measurement is made because the chunk-size curve beneath the tie is the single-host design result, and because the tie is hypothesis 1.

2.1 Configurations

R0. Raw file on XFS. QEMU's raw driver on the dedicated NVMe. The control, with no deduplication anywhere in the path.

R1. Zvol on ZFS 2.3 fast dedup. Its own pool on the same device, created and destroyed per run, opened by QEMU as a block device. `feature@fast_dedup; dedup=blake3`, because `dedup=on` hashes with SHA-256 regardless of the checksum property; `volblocksize=16K` as the primary arm and `4K` as the second; `compression=zle` so zero blocks do not collapse onto one DDT entry; `dedup_table_quota` unset and `zpool ddtprune` never run during a measurement; DDT memory from `zpool status -D`. OpenZFS direct IO applies neither to zvols nor with deduplication enabled, so R1 is ARC-backed in every arm and is reported as such.

R2. Raw file on XFS over dm-vdo (optional). Inline fixed-4 KiB deduplication in the kernel, mainline since 6.9, with its own XFS instance on the vdo device. Index memory from `vdo stats`.

R3. The backend on one host. Local store only, so `k` does not apply. Three chunk-size arms, below.

R0 against R3 is the cost of the daemon with everything else held constant. R1 is the deployed comparator

and differs in kernel boundary, caching, and allocation, so it is a case study beside the controlled pair, and deltas are attributed accordingly.

2.2 Chunk-size arms

Fixed 4 KiB chunks cost one index entry per 4 KiB: about 250 million entries per TB, about 10 GB of memory per TB at 40 bytes per entry, a 32-byte hash and an 8-byte offset. The alignment argument on page 00 predicts that they capture nearly every duplicate a Linux guest holds; the census measures the remainder. FastCDC with a 16 KiB mean cuts the index by four and loses an aligned 4 KiB match whenever the rest of its chunk differs. The one prior curve on VM images is Liquid's: 77% of bytes removed at 4 KiB, falling to 59% at 256 KiB on 183 images, with 256 KiB chosen for HDD seek cost; on NVMe the seek term is gone and the trade is index memory against capture.

Three arms: fixed 4 KiB, fixed 16 KiB, FastCDC 8 to 64 KiB with a 16 KiB mean. CDC boundaries snap to 4 KiB, so no guest block straddles two chunks and a 4 KiB overwrite invalidates one chunk, not two. Reported per arm: bytes stored, index bytes per TB, guest p99, write amplification, compactor CPU per GB. **Capture against index memory as a function of chunk size is the result this page produces.** The census below predicts the capture column for each arm before any run.

2.3 Workloads

- fio: 4 KiB random write and read at QD1 and QD32; 128 KiB sequential.
- Boot storm: N clones of one image booted together, N = 4, 16, 32; a clone is a copy of the manifest.
- Fleet replay: the synthetic fleet below written onto N guests, at two points on its timeline.
- Overwrite: a small SQLite database rewriting its pages in place for an hour, with guest discard on. This is the case where a store without reference counts leaks between sweeps and ZFS does not.

2.4 Metrics

- Guest p50 and p99 write and read latency against R0, compactor active and idle. Reported first.
- Bytes stored after compaction completes and the sweep has run, against the census prediction at the configuration's block size; bytes the sweep reclaimed reported beside it as the leak.
- Index or DDT bytes per stored TB.
- Write amplification: device bytes written per guest byte, from NVMe counters, with both legs (staging and store) reported, not one.
- Sustainable ingest, the point where the governor starts adding latency, and how much it adds.
- Chunk traffic against the settle window: chunks produced per guest byte written, on the overwrite workload.
- Compactor CPU per GB ingested, per chunk-size arm; Liquid gave hashing cost as its reason for large blocks, and here it is a number.
- Recovery: `kill -9, replay, fio --verify, FLUSH` covering writes on another queue; an empty discard; a daemon that stops answering.

2.5 Controls

Pinned vCPUs, performance governor, discarded warm-up, fresh filesystem or pool per repetition, at least five repetitions, variance beside every number. With `cache=none`, R0 and R2 have no host cache; `zfs_arc_max` on R1 and the daemon's cache size on R3 are set equal.

All configurations are observed at the guest boundary (fio's histograms, guest-side blktrace for the boot storm) plus host device counters. The daemon adds per-request stage timestamps drained to ndjson, cross-checked once against bpfttrace with the delta reported. `zpool` and `vdostats` figures are supplementary.

2.6 Prediction from a small census

A small census supplies the numbers the rest of the study is measured against: how many unique bytes the fleet holds under each arm's chunker, and how many bytes copy-on-write would already have shared.

Phase 0. `zdb -S` on a ZFS pool holding the cloned fleet. Pool traversal starts each dataset at its origin snapshot's transaction group, so blocks a clone inherited are counted once and the simulated ratio is duplicates beyond what clones already share; this reading of `dmu_traverse.c` is confirmed with a two-clone test before the number is cited.

The fleet. Ubuntu publishes dated cloud images and `snapshot.debian.org` serves the archive as of any date. An image installed as of T0 and upgraded monthly against the archive as of T1, T2, and on replays a real update history. N such clones with scripted drift (hostnames, logs, a few packages each) form the fleet. It is rebuilt by one command, dated, and is also the replay workload above.

The split. Per byte range: zero or unallocated (from the guest allocation map, excluded), unique, shared with the T0 base in place, duplicate at an aligned 4 KiB or 16 KiB boundary elsewhere in the fleet, or duplicate only at a shifted offset. The aligned columns predict R1 and the fixed arms. The CDC arm is predicted by running FastCDC with the arm's parameters over the images, because a 16 KiB mean chunk captures fewer aligned matches than 4 KiB blocks and more shifted ones, and the two effects do not add. Nothing further: no donors, no real fleets, no claims about time.

3 Multiple hosts

Part 2 runs the backend on two hosts with one parameter, k , and measures bytes moved and bytes stored against what `zfs send, rsync`, and two per-host ZFS pools would move and hold.

3.1 Two placement modes

k is the number of owners per chunk. The design supports any N ; the testbed has two hosts, so k takes two values, and they are two different experiments. With $k = 1$, a host that goes dark takes its chunks with it until it returns; a read that needs one waits or fails with an error, and nothing is lost if the disk comes back. Surviving a dark host at two hosts costs a full mirror of chunks ($k = 2$) plus fleet class for the staging tail.

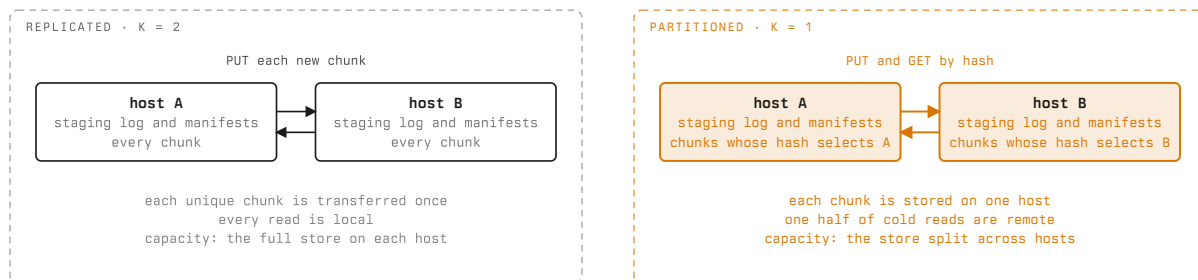


Figure 3: $k = 2$ provides transfer savings and keeps every read local. $k = 1$ provides capacity savings at the cost of remote reads. Two hosts with $k = 1$ send one half of cold reads to the peer in expectation, the largest share this testbed can produce.

3.2 Provisioning

A new guest on host B from an image whose chunks exist anywhere is a copy of the manifest: at least 32 bytes per chunk, about 80 MB for a 40 GB image at 16 KiB chunks. Every chunk it names already exists at its owner. In replicated mode no other data is transferred. In partitioned mode no other data is transferred either, because chunks are fetched on first read. **Provisioning cost is the size of the manifest.**

Baseline: `qemu-img convert` or `scp` of the raw file, and `zfs send | zfs recv` of the zvol, each moving the allocated size of the image. Liquid cloned by copying the metadata file and measured provisioning in seconds on 1 GbE, 8 GB to seven nodes in 730 s by `scp` and 35 s by Liquid; here it is bytes on the wire at 100 GbE.

3.3 Migration

To move a guest from A to B, the daemon freezes the device on A and takes E, hands the image to B by one fenced swap of its root record, ships the manifest and the staging extents in (D, E], and resumes on B. The root record names the writer and carries an epoch, and the swap is written durably on both hosts before B resumes; A accepts no write after the swap, and B resumes only after the swap names it. On resume the log is reconciled by evidence, the local high-water mark against the durable head, never by who claims to own it; in a prior implementation by the author, a refusal keyed on writer identity kept healthy guests from restarting. A 40 GB guest that compacted recently moves its manifest, about 80 MB at 16 KiB chunks, plus the staging tail, which was under 9 MB for an idle guest in that implementation and is workload-bound for a busy one.

Bytes are the small part of a migration. The disk cut measured 3 to 6 ms in that implementation and the rest of the blackout was orchestration, so the blackout is reported decomposed into freeze, swap, transfer, and resume, beside the bytes; governor pacing is disabled while the guest is paused. Memory migration is QEMU's and is out of scope; this is the disk.

Baseline: `rsync` of the raw file, and `zfs send` of the zvol. Since 2.0 `zfs send` emits no deduplicated stream, so the bytes are the logical size regardless of the DDT.

3.4 Synchronization after drift

Two guests, one on each host, are cloned from the same image and each updated independently to the same package set. Compaction on each host sends only the chunks the owner lacks, packed in sealed segments. Bytes on the wire are read against the census's unique-byte count for the pair. Chunks per second is reported beside bytes per second, because per-chunk cost is what caps a replication path before the link does. This is the apt upgrade case from page 00, measured.

3.5 Capacity

Partitioned mode stores each chunk once across the fleet. Measured: bytes on both stores after the fleet replay completes and the sweep has run, against two per-host ZFS pools holding the same guests, and index bytes on each host. Predicted: about half of the pools' bytes, and half of the index on each host. Also measured: the fraction of a guest's cold reads served by the other host. On two hosts with $k = 1$ that fraction is one half in expectation; in general it is $1 - k/N$, so a larger fleet at fixed k sends a larger share of its cold reads over the network, and the two-host number is a lower bound on that share.

3.6 Durability classes and their cost

In local class, between a FLUSH acknowledgment and the chunk being durable at its owner sits the compaction lag, (O, E] in the watermark's terms. It is reported in seconds under the fleet replay, as a distribution, with the segment size as the parameter. That window is what a lost host loses, and it is the RPO (recovery point objective) of local class.

In fleet class, the staging tail goes to the image's journal peer on every FLUSH and the acknowledgment waits for the peer's `fdatasync`, as Nutanix AOS and HPE SimpliVity do. The class costs one round trip plus one remote `fdatasync` per FLUSH, and it is measured as write p99 at QD1 against local class, on TCP, and on RDMA if the `ibverbs` arm lands. On the page 04 figures the transport is about a tenth of a cold read, which has 80 μ s of media beneath it; a FLUSH has no media time to hide the round trip behind, so the transport's share of it is far larger, and this row is where that share shows.

3.7 Measurements

Flow	Daemon	Baseline	Read against
provision	bytes transferred, both modes	scp of raw file; zfs send	manifest size
migrate	bytes transferred, both modes; blackout decomposed	rsync; zfs send	manifest size + staging tail; milliseconds for the cut
sync after drift	bytes and chunks per second sent by compaction	rsync; zfs send	census unique bytes
capacity	bytes stored, partitioned, after the sweep	two per-host ZFS pools	census prediction
index per host	index bytes on each host, both modes	DDT bytes per pool	k/N of the fleet index
remote fraction	cold reads served by the peer		one half in expectation
local-class window	seconds from ack to owner-durable		the RPO of local class
fleet-class cost	write p99 at QD1, TCP and RDMA	local class	one RTT plus one remote fdatsync per FLUSH

3.8 The locality objection

Dong et al. (FAST '11) rejected per-chunk hash placement for backup streams because it destroys read locality, and routed 1 MB super-chunks instead. This is primary storage with a local cache, so the fragmentation cost they argued about is measured directly on page 04. If it is large, placement by super-chunk is the knob, noted here and measured only if time remains.

4 Remote read

Part 3 measures the one place the network enters guest latency: a cold read whose chunk lives on another host. This page measures that read, then measures how much of it prefetch removes.

4.1 Where the time goes in a remote read

A 4 KiB random read at QD1 from an enterprise NVMe SSD completes in about 80 μ s: 81.6 μ s on a PM1725 in Systor '17 and about 80 μ s on a PM1735 in blk-switch; R0 measures the testbed's drive in week 1. On 100 GbE the transport sits on top. Against a null device, kernel nvme-rdma added 12.1 μ s and kernel nvme-tcp 21.4 μ s for a 4 KiB read on ConnectX-5 (SPDK 24.05); raw RDMA sits at 3 to 5 μ s (eRPC; SPDK); and on two CloudLab c6525-100g nodes, the testbed's node type, BPF-oF measured average round trips of 18 μ s over nvme-rdma and 30 μ s over nvme-tcp on kernel 5.12. A userspace daemon over kernel TCP has no published measurement as a remote read target; from the kernel TCP round-trip floor of 13 to 23 μ s (Homa; Zuo et al.) plus a file read, we estimate 20 to 30 μ s when it polls and more when it sleeps. The testbed replaces every one of these figures.

On these figures the difference between RDMA and TCP is about 9 μ s on a read of about 100 μ s. The larger factor, about 4x, is whether the chunk is in the owner's memory or on its disk. **If the figures hold, a chunk from a peer's memory over TCP arrives before one from local NVMe**, and hypothesis 3 tests this. Every host sends its reads of a chunk to the same k owners, so a chunk read by many guests is hot at its owner, and a remote read in that case is the memory row.

A caution from a prior implementation by the author: a peer round trip over QUIC with TLS on a bonded

25 GbE link measured 108 μ s at p50 and 257 μ s at p99 (unpublished); the daemon here uses kernel TCP with TCP_NODELAY on 100 GbE, and the number is measured rather than assumed.

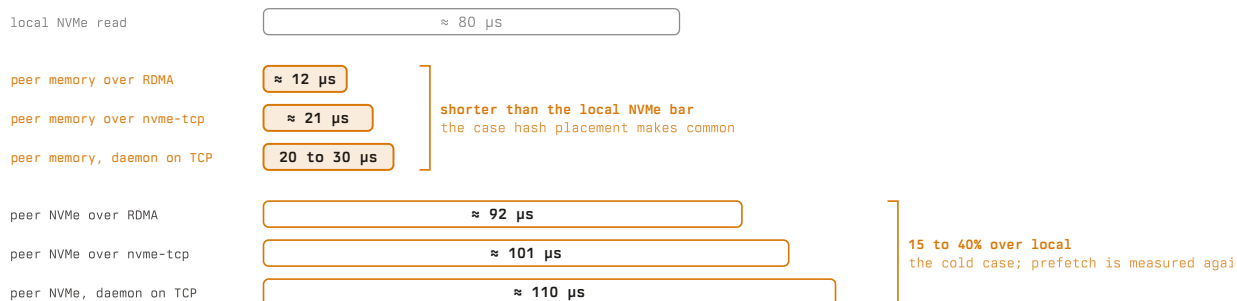


Figure 4: Literature values for one 4 KiB read at QD1, in microseconds, before the testbed measures them. The daemon rows are estimates.

4.2 Transport probes

The architecture's transport is the daemon over kernel TCP. The other rows exist to show what the kernel stack and the userspace hop each cost, and nothing depends on them.

Probe	What it isolates	Code
ib_read_lat -s 4096	the hardware floor	none
nvme-rdma export	kernel block path over RDMA; owner's store exported by nvmet as a file-backed namespace, buffered_io on for memory, off for media	configuration
nvme-tcp export	same over kernel TCP	configuration
daemon, TCP, busy-polling	the architecture, without the wakeup	the daemon
daemon, TCP, blocking	the architecture as deployed; the scheduler wakeup is the cost	the daemon
daemon, ibverbs two-sided[STRETCH]	the userspace hop without the kernel stack	~ 40 h

The nvmet export is a probe and not the architecture: it exposes the raw store, needs the reader to know offsets, and has no place for authentication. It is in the table because the difference between it and the daemon over the same TCP is the cost of the userspace hop, with SPDK's 1 μ s kernel-versus-userspace target delta as the reference point.

4.3 Method

- Same two hosts, NIC, drive, and kernel for every row. Kernel, firmware, MTU, IRQ affinity, interrupt moderation, C-states, busy-poll, and PFC state recorded.
- The link is measured before any remote number: `ib_read_lat` for the RDMA floor and a TCP ping-pong for the kernel floor, both recorded beside the rows.
- Two targets per row: a null device for fabric plus stack alone, and the real file for end to end. Each from the owner's memory and from its NVMe.
- Two load states for the file rows: quiet, and with PUT traffic running on its own connection at the ship rate from page 03, because a cold read in deployment competes with compaction. The difference is what the read-priority rule on page 01 buys.
- 4 KiB, 16 KiB, 64 KiB. p50, p99, p99.9. Five runs of 30 s, caches dropped between, medians with spread.

- QD sweep 1, 4, 16, 64 for throughput and CPU per IOPS on both ends; kernel TCP costs 2.5 to 3x the CPU of RDMA at equal IOPS in the SPDK 24.05 reports and in i10, and the ratio measured here is reported.
- RoCE hardware counters (`out_of_sequence`, `packet_seq_err`, `local_ack_timeout_err`) printed beside every RDMA number, showing zero retransmits on a fabric with no PFC.

4.4 Prefetch

The manifest tells the daemon what comes next. Depth sweep: sequential reads through the manifest with 1, 2, 4, 8, 16, 32 chunks in flight, at 4 KiB and 64 KiB. The bandwidth-delay point is about 250 KB for the fabric ($100 \text{ Gb/s} \times 20 \mu\text{s}$) and about 1.2 MB with media under it ($100 \text{ Gb/s} \times 100 \mu\text{s}$), so about 20 chunks of 64 KiB or 300 of 4 KiB in flight should hide the remote read entirely. Success is remote sequential throughput within the error bars of local.

Profile prefetch: record the chunk sequence of one boot, replay it on later boots. DADI, REAP, FaaSnap, VMTorrent, and Nydus each prefetch a recorded access profile, and DADI reports that this removes 95% of the gap between cold and warm start. It is a one-day implementation.

4.5 Under a guest workload

Partitioned boot storm at $N = 16$, with and without profile prefetch, against the same storm in replicated mode. Reported: guest p99, host device reads per guest byte, and the fraction of reads served by the peer, so the per-read cost and the miss rate can be multiplied. **The gap between partitioned with prefetch and replicated is the residual cost of one copy per chunk.**

4.6 The FLUSH round trip

Fleet class on page 03 puts one round trip and one remote `fdatsync` in front of every FLUSH acknowledgment, and there is no $80 \mu\text{s}$ of media to hide behind, so the round trip and the peer's `fdatsync` are the whole cost. It is measured here with the same discipline as the read rows: write p99 at QD1 for local class, for fleet class over the daemon on TCP, and for fleet class over `ibverbs` if that arm lands, with the peer's `fdatsync` time reported separately so the transport's share is visible.

4.7 RDMA on this testbed

The CloudLab fabric is lossy: no PFC or ECN is documented on the shared switches, and published work on this node type ran RoCE that way. Adaptive retransmission is enabled on the NIC and the counters above show whether the runs were clean. ConnectX-5 cannot do `io_uring` zero-copy receive, so that option is unavailable. None of this touches the architecture, which runs on kernel TCP and would run on any Ethernet.

4.8 Hypothesis 3, restated

- For a chunk not in the local cache, a read from the owner's memory arrives before the same read from local NVMe, on TCP and on RDMA.
- From the owner's NVMe it costs at most 40% over local on TCP and 15% on RDMA, at QD1, 4 KiB.
- At depth at or above the bandwidth-delay point, remote sequential throughput is within 10% of local.
- Partitioned boot storm p99 with profile prefetch is within 25% of replicated.

5 Plan

Fourteen weeks at about 320 hours, which is 23 a week against the 8 the course credit corresponds to. The plan is sized to the work, and the descoping order defines what is removed if it slips.

5.1 Hardware

Two CloudLab c6525-100g nodes (Utah), reserved as a pair. Per node: AMD EPYC 7402P, 24 cores at 2.80 GHz; 128 GB ECC DDR4-3200; two 1.6 TB PCIe 4.0 NVMe SSDs; ConnectX-5 Ex 100 GbE, one port on the experiment network. One NVMe holds the system and results, and the other is the device under test. The pair is one hop through a single switch.

RoCE between two of these nodes works on the lossy fabric: BPF-oF ran nvme-rdma and nvme-tcp between two c6525-100g nodes and measured 18 and 30 μ s average round trips. Self-built kernels are routine there; the Ubuntu 24.04 image ships 6.8, dm-vdo needs 6.9, and OpenZFS 2.3 is a source build, so a kernel and ZFS are built once in week 1 and snapshotted as an image. Reservations expire at 16 hours by default, so every run is scripted to complete inside one.

CloudLab is free for research. A project is opened by a faculty member and reviewed by CloudLab staff, so the sponsor opens it before Sep 9. Fallback: two OVHcloud Advance-4 2026 servers (EPYC 4585PX, 16 cores, 64 GB DDR5 ECC, 2 $\times$ 960 GB NVMe) on a 25 Gbps private link, which loses the RDMA arm and replaces the 100 GbE fabric with 25 GbE.

5.2 Schedule

Weeks	Build	Measure
1–2	vhost-user-blk daemon in passthrough: staging log, FLUSH, replay. Kernel and ZFS image.	R0, with the drive's read and fdatsync times; passthrough within 10% of R0 p99 (G1). Thresholds frozen. zdb -S phase 0 on the synthetic fleet.
3–5	Compactor with settle window, store, index, manifests, watermark, governor, recovery. Three chunk-size arms.	kill -9 recovery and the three ordering tests pass (G2). First capture numbers.
6–7	R1 configured, both volblocksize arms. R2 if time permits.	Part 1 table complete (G3), sweep before every capacity number.
8–9	Protocol with separate GET and PUT connections, rendezvous placement, k, segment PUT with durable ack, HAS, pins, surplus copies, sweep. Provisioning; migration with the fenced handoff.	Replicated mode on two nodes.
10	Partitioned mode. Fleet class over TCP.	Part 2 table complete (G4).
11–12	nvmet exports, RoCE configuration, busy-polling and blocking daemon, depth prefetch, profile prefetch.	Transport matrix and prefetch sweeps (G5). Partitioned boot storm.
13–14		Report; reproducibility pack (G6).

5.3 Gates

G1. Passthrough daemon under stock QEMU within 10% of R0 p99 by the end of week 2. If this slips, everything after it slips, and the sponsor is informed that week.

G2. kill -9 at arbitrary points, replay, fio --verify passes, before any daemon number is reported. Three ordering tests pass with it: a FLUSH covering writes completed on another queue, an empty discard, and a stalled daemon that is restarted with the guest still recoverable.

G3. Part 1 table complete: R0, R1 at two block sizes, R3 at three chunk sizes; latency, capture, index, amplification; variance beside every number.

G4. Part 2 table complete: both modes, every flow, bytes transferred against the census bound.

G5. Transport matrix complete for every non-stretch probe, null and file, memory and NVMe, with RoCE counters at zero.

G6. One command rebuilds the fleet from dated archives, and one command reruns every table on a fresh pair.

5.4 Descoping order

When the schedule slips, items come off from the top.

1. ibverbs daemon arm, and with it fleet class over RDMA.
2. Super-chunk placement.
3. R2 dm-vdo.
4. Profile prefetch (depth prefetch stays).
5. Fleet class over TCP. Hypothesis 4 is then reported as untested, with the literature's numbers as the estimate.
6. Partitioned mode. Replicated mode alone still gives hypothesis 2's transfer result.

Not removed under any slip: part 1, the nvmet TCP and RDMA probes, and the daemon over TCP.

5.5 Risks

- **Daemon overrun.** The largest risk and the reason G1 is at week 2. Protocol plumbing comes from maintained crates, so the hours go to the components listed as new code on page 01.
- **RoCE configuration.** GID selection, MTU, adaptive retransmission on a lossy fabric. Budgeted at 8 hours; if it exceeds 20, the RDMA rows are dropped and the TCP rows stand.
- **Node availability.** 36 nodes of this type exist, so the pair is reserved in week 1 for every measurement week.
- **Correctness debt.** The defects that stall or corrupt a guest are known in advance from a prior implementation: a FLUSH that misses a write completed on another queue, an empty discard that acknowledges a sequence number nothing wrote, a daemon that stops and leaves the guest in D-state. Each has a test in G2 and hours in weeks 3 to 5, before any number is taken.
- **O_DIRECT alignment.** A guest buffer not aligned to the logical block is bounced through an aligned copy, and the fraction of requests bounced is counted.
- **Known configuration pitfalls.** dedup=on means SHA-256; direct IO does nothing on zvols; the 100G interface stays down unless the profile declares a link on it.
- **Census realism.** Scripted drift is not real drift. The fleet is built from real dated archives, the scripts are published, and the numbers it supplies are bounds the daemon is read against, not claims about fleets in the wild.

5.6 Logistics

CS 4993, 1 credit. Expectations in writing before Sep 9. Thirty minutes of sponsor time every two weeks, with G1 as a scheduled meeting.

5.7 Future work

Availability. Fleet class is the seed of replication before acknowledgment; with it and $k \geq 2$ on $N \geq 3$ the system has a failure model, which needs membership, failure detection, and rebalancing, none of which this study touches.

Placement and reclamation. Super-chunk placement for locality; a cache policy that weighs a chunk's owner distance; an on-disk copy-on-read tier for chunks that are cold at their owner; and reference counts kept as derived state, with the sweep as the auditor, so an overwrite frees space at once as it does in ZFS.

The same split elsewhere. Prefix caching in LLM serving (vLLM, SGLang, Mooncake) names cached KV blocks by a hash chain over the whole token history, so two requests share only along a common prefix; that is lineage. The same document after two different preambles is computed twice; that is the cross-host case here, and its size on a real trace is unmeasured.

6 Prior art

Swept on 2026-09-01; sources and what was opened are in `docs/review/`. No system in the sweep combines a durable, sequence-numbered local write log with a stated FLUSH contract, a fleet-wide chunk store whose owners are the hosts themselves, a block device under a stock hypervisor, and a per-transport measurement of the remote cold read. Liquid (TPDS '14) is the nearest design and the row to read first; Datrium, Nutanix, and Fossil with Venti each share one component.

6.1 Nearest systems

Work	What it is	How this differs
Datrium DVX (2016), US20170031994A1	host-side fingerprinting, host flash as read cache, global deduplication on a shared data-node pool; the patent lists host-only ack as an alternative	peers as owners by hash instead of a shared pool; open implementation on stock QEMU; the cold read measured per transport
Nutanix AOS	local OpLog on SSD, mirrored to another node before ack; cluster-wide post-process deduplication at 16 KiB; per-node cache	no mirror on the write path by default, with the window measured and the mirror as fleet class; placement by hash instead of by vDisk locality; latency and capacity numbers, which its documentation does not give
Fossil + Venti (2002)	a disk write buffer in front of a content-addressed archive; the two-tier shape	block device under a VM instead of a filesystem; primary capacity instead of archival; more than one owner
Ceph + TiDedup (ATC '23)	post-process CDC into a chunk pool placed by CRUSH on the fingerprint; promotes on a cold miss	writes never cross the network; a host cache instead of promotion; a guest block path; latency numbers, which TiDedup does not report
vSAN ESA global deduplication (2025)	cluster-wide post-process 4 KiB deduplication, mirrored writes, 3 to 16 hosts, no published numbers	the per-host to cluster-wide change this study measures, with published numbers
HYDRAsTOR (FAST '09)	content-addressed blocks placed by DHT across a grid, global deduplication	secondary storage with network writes; no guest path
DeDe (ATC '09)	hosts hash in-band, deduplicate out-of-band against a shared index on a SAN, no coordinator	local disks instead of a SAN; chunks move to owners instead of pointers on shared storage

Work	What it is	How this differs
Liquid (TPDS '14)	FUSE file under a stock hypervisor; fixed 256 KiB to 1 MiB blocks hashed on flush or eviction from a 256 MB volatile write cache, pushed to range-partitioned data servers at VM shutdown; central meta server with refcounts; P2P Bloom-filter cache tier; copy-on-read disk cache; two replicas	a durable log with a FLUSH contract instead of a volatile buffer with no crash story; a vhost-user block device instead of FUSE; hosts as owners by rendezvous instead of a meta server and a data-server tier; exact HAS instead of Bloom filters; the miss cost measured, which Liquid names ("several times longer") and never measures

6.2 Remote fetch in prior systems

Work	What it measured	What it leaves open
Liquid (TPDS '14)	8 GB image to 7 nodes on 1 GbE: scp 730 s, NFS 510 s, BitTorrent 95 s, Liquid 35 s; on-demand boot 1.7x to 4x a cached boot; dedup 77% at 4 KiB falling to 59% at 256 KiB on 183 images	miss cost stated as "several times longer IO delay" and never measured; no latency numbers anywhere; HDD and 1 GbE
DADI (ATC '20)	block-level lazy loading with tree P2P; 10,000 containers on 1,000 hosts in 4 s; trace prefetch removes 95% of the cold gap; reads from a parent's page cache are faster than local disk	no per-read miss latency; not content-addressed
Slacker (FAST '16)	only 6.4% of a container image is read at startup; lazy fetch over NFS; run phase 17% slower	no per-block miss cost; centralized
VMTorrent (CoNEXT '12), VMThunder (TPDS '14)	demand-priority P2P VM image streaming with recorded profiles	startup seconds only
FaaSnap (EuroSys '22), REAP (ASPLOS '21)	lazy page faults from local disk at 13 μ s; userfaultfd over 128 μ s uncached; working set 9% of footprint	memory, not disk; local
SnowFlock (EuroSys '09)	275 μ s per page fetched over gigabit, 82% of it in the network stack	the only in-VM remote per-unit number, and it is from 2009
Dahlin et al. (OSDI '94)	cooperative caching: remote client memory at 1.25 ms against disk at 15 ms; N-chance forwarding	the argument this study repeats at 100 GbE with content-addressed chunks
CLB (VEE '17), Satori (ATC '09)	content-keyed sharing of VM disk reads across guests on one host; 95 to 98% of boot reads eliminated	single host; no store

Among the systems above, which span FAST, ATC, OSDI, NSDI, EuroSys, ASPLOS, CoNEXT, VEE, and TPDS from 1994 to 2022, none reports the latency of a content-addressed chunk fetched from a peer inside a VM block read path. DADI, VMTorrent, VMThunder, REAP, and FaaSnap report startup time in seconds and hide the per-read penalty behind a recorded access profile; Liquid names the penalty and does not measure it.

6.3 Transport measurements in prior work

i10 (NSDI '20) and blk-switch (OSDI '21) showed kernel TCP can match RDMA on throughput per core with batching, at a latency cost of 50 to 100 μ s at low load. The SPDK 24.05 reports on ConnectX-5 put kernel nvme-rdma at 12.1 μ s and kernel nvme-tcp at 21.4 μ s for a 4 KiB read against a null device. BPF-oF (2023) measured 18 μ s over nvme-rdma and 30 μ s over nvme-tcp between two CloudLab c6525-100g nodes, the testbed's node type, on kernel 5.12. Homa (ATC '21) and eRPC (NSDI '19) put kernel bypass at 2 to 4 μ s and attribute the rest of kernel TCP to wakeups and core selection. No storage paper in the sweep measured a blocking userspace daemon over kernel TCP as a remote read target; that row is estimated on page 04 and measured here.

6.4 Objections already in print

Dong et al. (FAST '11) rejected per-chunk hash placement for backup streams on locality grounds and routed 1 MB super-chunks; page 03 answers with a local cache and page 04 measures the cost. **Meyer and Bolosky** (FAST '11) found that deduplication savings grow with the log of the number of machines in one domain, on 857 desktops; two hosts is the floor of the capacity half of hypothesis 2, and a larger fleet gains more. **Jin and Miller** (SYSTOR '09) found fixed blocks match CDC on VM images, which is why part 1 predicts a tie. **despairlabs** (2024) tells ZFS operators to use clones and block cloning for the copy case and deduplication rarely; hypothesis 1 predicts agreement on one host, and hypothesis 2 measures the cross-host case that advice does not address. **The hyperconverged products** in the table (Nutanix, Datrium, SimpliVity, vSAN ESA) mirror a write over the network before acknowledging it, so a local-only acknowledgment is a durability trade rather than a free latency gain; page 01 makes it a class and page 03 prices both.

6.5 What this study adds

Datrium's patent and Nutanix's design are cited by name, Liquid is the nearest prior system, and Fossil and Venti are the origin of the two-tier shape. The study's contribution is the measurement: what content addressing provides across hosts on commodity hardware under a stock hypervisor, and what the remote cold read costs, per transport.